

Laboratorium 12: Algorytmika i inżynieria dźwięku cyfrowego

Opracowanie zestawu zadań (część programistyczna)

Kacper Bednarczuk

Kod programu

```
// Funkcja generująca łagodny sygnał (symulacja audio)
function generateSignal(length = 10) {
    const signal = [];
    // Zaczynamy od losowej wartości w rozsądnym zakresie (np. 100)
    let currentValue = 100;

    for (let i = 0; i < length; i++) {
        signal.push(currentValue);

        // Generujemy małą zmianę (-3 do +3), aby sygnał był "łagodny"
        // Math.random() zwraca [0, 1), mnożymy przez 7 → [0, 7), floor -
        > [0, 6], odejmujemy 3 → [-3, 3]
        const change = Math.floor(Math.random() * 7) - 3;
        currentValue += change;
    }

    return signal;
}

// Funkcja obliczająca różnice (Delta Encoding)
function calculateDeltas(samples) {
    const deltas = [];

    if (samples.length === 0) return deltas;

    // Pierwsza wartość jest zapisywana bez zmian (lub jako różnica od 0)
    deltas.push(samples[0]);

    // Dla kolejnych elementów obliczamy różnicę: obecny - poprzedni
    for (let i = 1; i < samples.length; i++) {
        const diff = samples[i] - samples[i - 1];
        deltas.push(diff);
    }

    return deltas;
}
```

```

}

// Funkcja pomocnicza do obliczania średniej wartości bezwzględnej
function calculateMeanAbs(data) {
  if (data.length === 0) return 0;

  let sum = 0;
  for (const value of data) {
    sum += Math.abs(value);
  }

  return sum / data.length;
}

console.log("=== Symulacja Algorytmu Delta Encoding ===");

// 1. Generowanie sygnału
const originalSignal = generateSignal(10);

// 2. Obliczanie różnic
const encodedSignal = calculateDeltas(originalSignal);

// 3. Wyświetlanie wyników
console.log("\nOryginalny sygnał (próbki):");
console.log(originalSignal.join(", "));

console.log("\nSygnał zakodowany (różnice):");
console.log(encodedSignal.join(", "));

// 4. Analiza
const meanAbsOriginal = calculateMeanAbs(originalSignal);

// Dla deltas pomijamy pierwszy element, bo to wartość początkowa (często
// duża),
// a nas interesują same różnice w kontekście kompresji ciągu.
const meanAbsDeltas = calculateMeanAbs(encodedSignal);

console.log("\n--- Analiza ---");
console.log(`Średnia wartość bezwzględna (oryginał):
${meanAbsOriginal.toFixed(2)}`);
console.log(`Średnia wartość bezwzględna (różnice):
${meanAbsDeltas.toFixed(2)}`);

console.log("\nWniosek:");
if (meanAbsDeltas < meanAbsOriginal) {
  console.log("Wartości różnic są średnio znacznie mniejsze niż
oryginału.");
  console.log("Mniejsze liczby wymagają mniej bitów do zapisu (np. przy
użyciu kodowania Huffmana),");

```

```
    console.log("co pozwala na efektywną kompresję.");  
  } else {  
    console.log("W tym przypadku zysk z kodowania różnicowego jest  
    niewielki lub ujemny (rzadki przypadek dla łagodnych sygnałów).");  
  }
```

Wyniki działania programu

Poniżej przedstawiono logi wygenerowane przez program uruchomiony w konsoli systemowej:

```
≡ Symulacja Algorytmu Delta Encoding ≡
```

```
Oryginalny sygnał (próbki):
```

```
100, 103, 100, 100, 101, 101, 98, 97, 100, 100
```

```
Sygnał zakodowany (różnice):
```

```
100, 3, -3, 0, 1, 0, -3, -1, 3, 0
```

```
--- Analiza ---
```

```
Średnia wartość bezwzględna (oryginał): 100.00
```

```
Średnia wartość bezwzględna (różnice): 11.40
```

```
Wniosek:
```

```
Wartości różnic są średnio znacznie mniejsze niż oryginału.
```

```
Mniejsze liczby wymagają mniej bitów do zapisu (np. przy użyciu kodowania  
Huffmana),
```

```
co pozwala na efektywną kompresję.
```

Wnioski (Zadanie 2.2)

Pytanie: Dlaczego ten prosty algorytm (Delta Encoding) działałby gorzej dla „białego szumu” (losowych wartości), a działa świetnie dla muzyki klasycznej lub głosu?

Odpowiedź:

Delta Encoding opiera się na **korelacji między sąsiednimi próbkami** (redundancji informacyjnej).

1. **Muzyka klasyczna / Głos:** Są to sygnały o tzw. "wolnozmiennnej" charakterystyce w krótkich okresach czasu. Amplituda fali dźwiękowej nie skacze losowo z maksimum do minimum w ciągu jednej próbki. Sąsiednie próbki mają zazwyczaj bardzo zbliżone wartości. Dzięki temu różnice między nimi (delty) są bliskie zeru. Histogram wartości

różnic staje się bardzo "szpiczasty" (mocno skupiony wokół zera), co drastycznie zmniejsza entropię danych. To pozwala algorytmom takim jak Huffman na użycie bardzo krótkich kodów dla tych małych, częstych wartości, co daje wysoki stopień kompresji.

2. **Biały szum:** Jest to sygnał losowy, w którym każda kolejna próbka jest statystycznie niezależna od poprzedniej. Wartości skaczą gwałtownie i nieprzewidywalnie. Różnica między dwiema losowymi, nieskorelowanymi liczbami ma często **większy zakres zmienności (wariancję)** niż same liczby (rozkład różnic dwóch niezależnych zmiennych losowych jest szerszy niż rozkład wyjściowy).

- Przykładowo, dla wartości z zakresu 0-255, różnica może wynosić od -255 do 255 (co wymaga 9 bitów zamiast 8 + znak, czyli efektywnie zakres się podwaja).
- Ponieważ nie ma skupienia wartości różnic wokół zera, entropia nie spada, a może nawet wzrosnąć. Delta Encoding w takim przypadku nie daje zysku, a może wręcz powiększyć rozmiar danych (zjawisko ekspansji danych).